



RSL POS

Development, Testing & Deployment Guide

Practical guide for the Ryantech software engineering attachés

1. Purpose

Build a simple, reliable and easy-to-use Point-of-Sale system for small and medium retail shops in Nairobi. The target businesses include clothing shops, shops selling moving boxes and packaging materials, imported/general merchandise shops, boutiques and similar retailers.

The objective is to produce a working MVP that can be demonstrated, tested with a real shop workflow and deployed. Do not spend weeks on research before development. Make reasonable assumptions, document them, build the MVP and validate it as you go.

The Ryantech attachment programme already identifies a POS as a suitable product covering sales, stock, receipts, customers, cashier accounts and reports. □filecite□turn0file0□L64-L81□

2. What We Are Building

RSL POS should initially provide these core functions:

Module	Initial Scope
Login & Users	Admin/owner and cashier accounts with basic permissions.
Dashboard	Today's sales, number of transactions, low-stock items and simple business summary.

RYANTECH SOLUTIONS LTD.

Office: Ground Floor, Luther Plaza, Nyerere Road, Nairobi, Kenya.

WhatsApp/Call: +254 720 475 664

Website: www.ryantech.co.ke | **Email:** info@ryantech.co.ke

Products	Add/edit products, category, SKU/code, buying price, selling price, quantity and optional barcode.
POS / Sales	Search/select products, add to cart, change quantity, discount, calculate total and complete sale.
Payments	Cash and M-Pesa as initial payment methods; structure code so more methods can be added.
Receipts	Generate a printable/downloadable receipt and store the sale.
Inventory	Stock in, stock out, adjustments, current quantity and low-stock indication.
Customers	Basic customer records and purchase history where required.
Suppliers	Basic supplier records and purchase/stock-receiving records.
Returns	Record returned items and update stock.
Reports	Daily/weekly/monthly sales, product sales, payment summary and stock report.

3. Development Plan

Follow the phases below. Complete each phase before moving to the next.

Phase 1 — Setup: Create GitHub repository, project structure, development environment, database connection and README.

RYANTECH SOLUTIONS LTD.

Office: Ground Floor. Luther Plaza, Nyerere Road, Nairobi, Kenya.

WhatsApp/Call: +254 720 475 664

Website: www.ryantech.co.ke | Email: info@ryantech.co.ke

Phase 2 — Database: Design tables and relationships for users, products, categories, sales, sale items, payments, stock movements, customers and suppliers.

Phase 3 — Authentication: Implement login, user roles and protected pages/API endpoints.

Phase 4 — Products & Inventory: Build product management, categories, stock receiving, adjustments and low-stock logic.

Phase 5 — POS Sales: Build the cashier screen: product search, cart, quantities, discounts, totals and sale completion.

Phase 6 — Payments & Receipts: Record payment method, prepare M-Pesa integration structure and generate printable receipts.

Phase 7 — Customers, Suppliers & Returns: Add the supporting records and workflows needed by the MVP.

Phase 8 — Reports: Create useful sales, stock, product and payment reports.

Phase 9 — Testing: Test each workflow, fix errors and test with realistic sample data.

Phase 10 — Deployment: Deploy the application, configure production database/environment variables and create a demo account.

Phase 11 — Documentation & Demo: Update README/user guide, demonstrate the complete workflow and record remaining improvements.

4. Suggested Technical Approach

Use technologies the team can maintain and that fit Ryantech's existing development environment. The following is a practical starting point; the team may adjust it after confirming requirements.

- Frontend: React or another modern web frontend the team is comfortable maintaining.
- Backend: FastAPI/Python or another approved Ryantech backend framework.
- Database: PostgreSQL.
- API: REST API between frontend and backend.
- Authentication: Secure token/session-based authentication with role checks.
- Version control: Git + GitHub.
- Deployment: A subdomain under ryantech.co.ke where practical.

RYANTECH SOLUTIONS LTD.

Office: Ground Floor, Luther Plaza, Nyerere Road, Nairobi, Kenya.

WhatsApp/Call: +254 720 475 664

Website: www.ryantech.co.ke | Email: info@ryantech.co.ke

The attachment programme requires Git/GitHub, documented projects, deployment where practical and source code under Ryantech's GitHub organisation. □filecite□turn0file0□L22-L29□

5. Minimum Database Structure

Start with a clean relational model. Suggested core tables:

- users — name, username/email, password hash, role, status
- categories — category name
- products — name, SKU, barcode, category, buying price, selling price, quantity, reorder level
- suppliers — supplier details
- purchases — supplier, date, total, status
- purchase_items — product, quantity, unit cost
- customers — basic customer details
- sales — cashier, customer, date/time, subtotal, discount, total, status
- sale_items — sale, product, quantity, unit price, discount
- payments — sale, method, amount, reference/status
- stock_movements — product, movement type, quantity, reference, user, date/time

6. Main POS Workflow

1. Cashier logs in.
2. Cashier searches/scans a product.
3. Product is added to the cart.
4. Cashier changes quantity or applies an allowed discount.
5. System calculates subtotal and total.
6. Customer chooses payment method.
7. Payment is recorded/confirmed.
8. System saves the sale and sale items.
9. Stock is automatically reduced.
10. Receipt is generated.
11. Sale appears in reports.

7. Stock Receiving Workflow

12. Admin/authorized user selects supplier.
13. User records products received and quantities.

RYANTECH SOLUTIONS LTD.

Office: Ground Floor, Luther Plaza, Nyerere Road, Nairobi, Kenya.

WhatsApp/Call: +254 720 475 664

Website: www.ryantech.co.ke | Email: info@ryantech.co.ke

14. System records purchase cost.
15. Stock quantities increase.
16. Stock movement is recorded for audit purposes.
17. Purchase is available in purchase history.

8. Important Business Rules

- A completed sale must have at least one sale item.
- Stock should not become negative unless an administrator explicitly enables that behaviour.
- Every stock change should create a stock movement record.
- Every sale should identify the cashier/user who processed it.
- Do not delete completed sales; use cancellation/return procedures so records remain auditable.
- Product selling price and buying price should be stored separately.
- Payment records should be linked to the relevant sale.
- Sensitive credentials and API keys must never be committed to GitHub.

9. M-Pesa Integration

Do not make M-Pesa integration the first blocker. Build the POS so payment methods are modular. First make cash sales work correctly, then implement/test the M-Pesa workflow.

- Create a payment record for every sale.
- Support payment method = Cash initially.
- Prepare payment method = M-Pesa with reference/transaction status fields.
- When integrating M-Pesa, handle pending, successful and failed transactions.
- Never store sensitive credentials directly in source code; use environment variables.
- Test payment failures and duplicate callbacks/confirmations before production.

10. Minimum Screens

- Login
- Dashboard
- POS / New Sale
- Products
- Add/Edit Product
- Inventory / Stock Movements
- Purchases / Receive Stock

RYANTECH SOLUTIONS LTD.

Office: Ground Floor. Luther Plaza, Nyerere Road, Nairobi, Kenya.

WhatsApp/Call: +254 720 475 664

Website: www.ryantech.co.ke | **Email:** info@ryantech.co.ke

- Customers
- Suppliers
- Sales History / Sale Details
- Returns
- Reports
- Users & Settings

11. Testing Checklist

- Login with correct and incorrect credentials.
- Confirm cashier cannot access restricted admin functions.
- Create, edit and search products.
- Receive stock and verify quantity increases.
- Make a complete sale and verify stock decreases.
- Test multiple quantities of the same product.
- Test discounts and totals.
- Test cash payment.
- Test receipt generation/printing.
- Test return and verify stock is adjusted correctly.
- Test low-stock indication.
- Verify sales reports against known sample transactions.
- Test invalid inputs and duplicate actions.
- Test on both desktop and mobile-sized screens.

12. Deployment Guide

18. Create the production database.
19. Set production environment variables: database URL, secret keys and integration credentials.
20. Run database migrations.
21. Create the production admin account securely.
22. Build and deploy the frontend.
23. Deploy the backend/API.
24. Configure the domain/subdomain and HTTPS.
25. Test login, product creation, sale, stock update, receipt and reports in production.
26. Create a safe demo/test account where appropriate.
27. Document the deployment and rollback/update procedure.

RYANTECH SOLUTIONS LTD.

Office: Ground Floor, Luther Plaza, Nyerere Road, Nairobi, Kenya.

WhatsApp/Call: +254 720 475 664

Website: www.ryantech.co.ke | Email: info@ryantech.co.ke

The attachment programme specifies a preferred deployment under ryantech.co.ke, source code under Ryantech's GitHub organisation, a README covering setup/usage/deployment and a demo/test account where appropriate. [filecite0turn0file0L22-L29](#)

14. GitHub & Team Working

- Create one repository for the RSL POS project.
- Use branches for features where practical.
- Use meaningful commit messages, e.g. `feat: add product management`.
- Open pull requests for reviewed changes.
- Do not commit `.env` files, passwords, API keys or other secrets.
- Keep the README updated.
- Review code before merging into the main branch.

These practices directly follow the attachment programme's GitHub and code-quality requirements. [filecite0turn0file0L134-L143](#)

15. Suggested Attaché Team Roles

Role	Main Responsibility
Backend/Database	API, business logic, database and integrations
Frontend	POS screen, dashboard, forms and reports UI
QA/Testing	Test cases, bug tracking and regression testing
Documentation/Deployment	README, user guide, deployment and demo environment

Roles may rotate. The attachment programme encourages small product teams and reusable components rather than completely unrelated individual projects. [filecite0turn0file0L94-L99](#)

16. What They Must Deliver

- Working RSL POS MVP.

RYANTECH SOLUTIONS LTD.

Office: Ground Floor. Luther Plaza, Nyerere Road, Nairobi, Kenya.

WhatsApp/Call: +254 720 475 664

Website: www.ryantech.co.ke | Email: info@ryantech.co.ke

- GitHub repository with clean, understandable code.
- Production/deployed demo where practical.
- Database schema/migrations.
- README with setup, usage and deployment instructions.
- Basic user guide.
- Test cases and evidence of testing.
- Final presentation and live demonstration.

17. Development Principle

Do not overbuild the first version.

Get the basic flow working first: LOGIN → PRODUCTS → STOCK → SALE → PAYMENT → RECEIPT → REPORT. Once this works reliably, add returns, suppliers, customers, M-Pesa integration, advanced reports and other features.

LEARN → BUILD → TEST → DEPLOY → DEMONSTRATE → IMPROVE

RYANTECH SOLUTIONS LTD.

Office: Ground Floor, Luther Plaza, Nyerere Road, Nairobi, Kenya.

WhatsApp/Call: +254 720 475 664

Website: www.ryantech.co.ke | Email: info@ryantech.co.ke